PKG 10: COMPARISON 02 - compareTo AND SORTING

```java
public class CompareToIntro03 {

    public static void main(String[] args) {

        System.out.println("=== equals():");
        System.out.println("AB".equals("AB"));
        System.out.println("AB".equals("BC"));

        System.out.println("=== compareTo():");
        System.out.println("AB".compareTo("BC"));
        System.out.println("BC".compareTo("BC"));
        System.out.println("CD".compareTo("BC"));

        System.out.println("===");

        String a = "a random string";
        String b = "another string ";

        System.out.println(b.compareTo(a));
    }
}
```

```
=== equals():
true
false
=== compareTo():
-1
0
1
===
78
```

```java
import java.util.*;

class MovieZ implements Comparable<MovieZ> {

    private final double rating;
    private final String name;
    private final int year;

    @Override
    public int compareTo(MovieZ m) {
        return this.year - m.year;
    }
    // Code in the box →
}
```

```java
public MovieZ(String nm, double rt, int yr) {
    this.name = nm;
    this.rating = rt;
    this.year = yr;
}
public double getRating() {
    return rating;
}
public String getName() {
    return name;
}
public int getYear() {
    return year;
}
```

```java
class MoviesByYearDriver {

    public static void main(String[] args) {
        ArrayList<MovieZ> list = new ArrayList<>();
        list.add(new MovieZ("Force Awakens", 8.3, 2015));
        list.add(new MovieZ("Star Wars", 8.7, 1977));
        list.add(new MovieZ("Empire Strikes Back", 8.8, 1980));
        list.add(new MovieZ("Return of the Jedi", 8.4, 1983));

        Collections.sort(list);

        System.out.println("Movies after sorting : ");
        list.forEach((movie) -> {
            System.out.println(movie.getName() + " "
                    + movie.getRating() + " "
                    + movie.getYear());
        });
    }
}
```

```
Movies after sorting:
Star Wars 8.7 1977
Empire Strikes Back 8.8 1980
Return of the Jedi 8.4 1983
Force Awakens 8.3 2015
```

COMPARISON

```java
import java.util.*;

class Computer implements Comparable<Computer> {

    private final String name;
    private final double cpuSpeed;

    @Override
    public int compareTo(Computer computer) {
        int result = (int) ((this.cpuSpeed - computer.cpuSpeed) * 10);
        return result;
    }

    public Computer(String name, double cpuSpeed) {
        this.name = name;
        this.cpuSpeed = cpuSpeed;
    }

    public String getName() {
        return name;
    }

    public double getCpuSpeed() {
        return cpuSpeed;
    }
}

class DesktopX extends Computer {

    public DesktopX(String name, double cpuSpeed) {
        super(name, cpuSpeed);
    }

}

class LaptopX extends Computer {

    public LaptopX(String name, double cpuSpeed) {
        super(name, cpuSpeed);
    }

}

class ComputerDriver {

    public static void main(String[] args) {

        ArrayList<DesktopX> list = new ArrayList<>();
        list.add(new DesktopX("Dell XT7500", 3.5));
        list.add(new DesktopX("Dell XPS910", 2.8));
        list.add(new DesktopX("Dell T9000", 3.8));
        list.add(new DesktopX("Dell D950", 1.5));

        Collections.sort(list);

        System.out.println("Desktops after sorting : ");
        list.forEach((desktop) -> {
            System.out.println(desktop.getCpuSpeed() + " " + desktop.getName());
        });

        System.out.println("========");
```

```
Desktops after sorting:
1.5 Dell D950
2.8 Dell XPS910
3.5 Dell XT7500
3.8 Dell T9000
========
Laptops after sorting:
1.9 Apple I9
2.2 Apple E6
2.4 Apple T5
3.7 Apple X1
========
Desktops/Laptops after
sorting:
1.0 Apple E6
1.5 Dell E1
1.5 Apple E7
2.0 Dell E2
2.0 Apple E8
2.5 Dell E3
2.5 Apple E9
3.0 Dell E4
3.0 Apple E10
3.5 Dell E5
========
Desktops/Laptops after
sorting:
1.0 Apple E6
1.0 SONY E10
1.5 Dell E1
1.5 Apple E7
2.0 Dell E2
2.0 Apple E8
2.0 SONY E10
2.5 Dell E3
2.5 Apple E9
3.0 Dell E4
3.0 Apple E10
3.0 SONY E10
3.5 Dell E5
4.0 SONY E10
5.0 SONY E10
```

```java

        ArrayList<LaptopX> list02 = new ArrayList<>();
        list02.add(new LaptopX("Apple X1", 3.7));
        list02.add(new LaptopX("Apple T5", 2.4));
        list02.add(new LaptopX("Apple E6", 2.2));
        list02.add(new LaptopX("Apple I9", 1.9));

        Collections.sort(list02);

        System.out.println("Laptops after sorting : ");
        list02.forEach((laptop) -> {
            System.out.println(laptop.getCpuSpeed() + " " + laptop.getName());
        });

        System.out.println("========");

        ArrayList<Computer> list03 = new ArrayList<>();
        list03.add(new DesktopX("Dell E4", 3.0));
        list03.add(new DesktopX("Dell E1", 1.5));
        list03.add(new DesktopX("Dell E2", 2.0));
        list03.add(new DesktopX("Dell E5", 3.5));
        list03.add(new DesktopX("Dell E3", 2.5));
        list03.add(new LaptopX("Apple E7", 1.5));
        list03.add(new LaptopX("Apple E8", 2.0));
        list03.add(new LaptopX("Apple E6", 1.0));
        list03.add(new LaptopX("Apple E9", 2.5));
        list03.add(new LaptopX("Apple E10", 3.0));

        Collections.sort(list03);

        System.out.println("Desktops/Laptops after sorting : ");
        list03.forEach((machine) -> {
            System.out.println(machine.getCpuSpeed() + " " + machine.getName());
        });

        System.out.println("========");

        ArrayList<Computer> list04 = new ArrayList<>();
        list04.add(new DesktopX("Dell E4", 3.0));
        list04.add(new DesktopX("Dell E1", 1.5));
        list04.add(new DesktopX("Dell E2", 2.0));
        list04.add(new DesktopX("Dell E5", 3.5));
        list04.add(new DesktopX("Dell E3", 2.5));
        list04.add(new LaptopX("Apple E7", 1.5));
        list04.add(new LaptopX("Apple E8", 2.0));
        list04.add(new LaptopX("Apple E6", 1.0));
        list04.add(new LaptopX("Apple E9", 2.5));
        list04.add(new LaptopX("Apple E10", 3.0));
        list04.add(new Computer("SONY E10", 1.0));
        list04.add(new Computer("SONY E10", 2.0));
        list04.add(new Computer("SONY E10", 3.0));
        list04.add(new Computer("SONY E10", 4.0));
        list04.add(new Computer("SONY E10", 5.0));

        Collections.sort(list04);

        System.out.println("Desktops/Laptops after sorting: ");
        list04.forEach((computer) -> {
            System.out.println(computer.getCpuSpeed() + " " + computer.getName());
        });

    }
}
```

```
public class SelectionSort {

    public static <T extends Comparable<? super T>>
            void selectionSort(T[] a, int n) {
        for (int index = 0; index < n - 1; index++) {
            int indexOfNextSmallest = getIndexOfSmallest(a, index, n - 1);
            swap(a, index, indexOfNextSmallest);
            // Assertion: a[0] <= a[1] <= . . . <= a[index] <= all other a[i]
        } // end for
    } // end selectionSort

    // Finds the index of the smallest value in a portion of an array a.
    // Precondition: a.length > last >= first >= 0.
    // Returns the index of the smallest value among
    // a[first], a[first + 1], . . . , a[last].
    private static <T extends Comparable<? super T>>
            int getIndexOfSmallest(T[] a, int first, int last) {
        T min = a[first];
        int indexOfMin = first;
        for (int index = first + 1; index <= last; index++) {
            if (a[index].compareTo(min) < 0) {
                min = a[index];
                indexOfMin = index;
            } // end if
            // Assertion: min is the smallest of a[first] through a[index].
        } // end for

        return indexOfMin;
    } // end getIndexOfSmallest

    // Swaps the array entries a[i] and a[j].
    private static void swap(Object[] a, int i, int j) {
        Object temp = a[i];
        a[i] = a[j];
        a[j] = temp;
    } // end swap
} // end SelectionSort
```

```java
public class SelectionSortRecursive {

    public static <T extends Comparable<? super T>>
            void selectionSortRecursive(T[] a, int n) {
        selectionSortRecursive(a, 0, n - 1); // invoke recursive method
    } // end selectionSort


    public static <T extends Comparable<? super T>>
            void selectionSortRecursive(T[] a, int first, int last) {
        if (first < last) {  // Place the smallest value at beginning of array:
            int indexOfNextSmallest = getIndexOfSmallest(a, first, last);
            swap(a, first, indexOfNextSmallest);
            selectionSortRecursive(a, first + 1, last);
        } // end if
    } // end selectionSort


    // Returns the index of the smallest value in a portion of an array.
    private static <T extends Comparable<? super T>>
            int getIndexOfSmallest(T[] a, int first, int last) {
        T min = a[first];
        int indexOfMin = first;
        for (int index = first + 1; index <= last; index++) {
            if (a[index].compareTo(min) < 0) {
                min = a[index];
                indexOfMin = index;
                // Assertion: min is the smallest of a[first] through a[index].
            } // end if
        } // end for
        return indexOfMin;
    } // end getIndexOfSmallest


    // Swaps the array entries a[i] and a[j].
    private static void swap(Object[] a, int i, int j) {
        Object temp = a[i];
        a[i] = a[j];
        a[j] = temp;
    } // end swap

} // end SelectionSort
```

SORTING

```java
public class InsertionSort {
    public static <T extends Comparable<? super T>>
            void insertionSort(T[] a, int n) {
        insertionSort(a, 0, n - 1);
    } // end insertionSort

    public static <T extends Comparable<? super T>>
            void insertionSort(T[] a, int first, int last) {
        for (int unsorted = first + 1; unsorted <= last; unsorted++) {
            // Assertion: a[first] <= a[first + 1] <= ... <= a[unsorted - 1]

            T firstUnsorted = a[unsorted];

            insertInOrder(firstUnsorted, a, first, unsorted - 1);
        } // end for
    } // end insertionSort

    private static <T extends Comparable<? super T>>
            void insertInOrder(T anEntry, T[] a, int begin, int end) {
        int index = end;

        while ((index >= begin) && (anEntry.compareTo(a[index]) < 0)) {
            a[index + 1] = a[index]; // Make room
            index--;
        } // end for

        // Assertion: a[index + 1] is available
        a[index + 1] = anEntry;  // Insert
    } // end insertInOrder
} // end InsertionSort
```

```java
public class InsertionSortRecursive {
    public static <T extends Comparable<? super T>>
            void insertionSortRecursive(T[] a, int n) {
        insertionSortRecursive(a, 0, n - 1);
    } // end insertionSort

    public static <T extends Comparable<? super T>>
            void insertionSortRecursive(T[] a, int first, int last) {
        if (first < last) {
            // Sort all but the last entry
            insertionSortRecursive(a, first, last - 1);

            // Insert the last entry in sorted order
            insertInOrder(a[last], a, first, last - 1);
        } // end if
    } // end insertionSort

    // Inserts element into the sorted array elements a[begin] through a[end].
    private static <T extends Comparable<? super T>>
            void insertInOrder(T element, T[] a, int begin, int end) {
        if (element.compareTo(a[end]) >= 0) {
            a[end + 1] = element;
        } else if (begin < end) {
            a[end + 1] = a[end];
            insertInOrder(element, a, begin, end - 1);
        } else {
            a[end + 1] = a[end];
            a[end] = element;
        } // end if
    } // end insertInOrder
} // end InsertionSortRecursive
```

```java
public class ShellSort {

    public static <T extends Comparable<? super T>>
            void shellSort(T[] a, int n) {
        shellSort(a, 0, n - 1);
    } // end shellSort

    public static <T extends Comparable<? super T>>
            void shellSort(T[] a, int first, int last) {
        int n = last - first + 1; // Number of array entries
        int space = n / 2;
        while (space > 0) {
            for (int begin = first; begin < first + space; begin++) {
                incrementalInsertionSort(a, begin, last, space);
            } // end for
            space = space / 2;
        } // end while
    } // end shellSort

    private static <T extends Comparable<? super T>>
            void incrementalInsertionSort(T[] a, int first, int last, int space) {
        int unsorted, index;

        for (unsorted = first + space; unsorted <= last;
                unsorted = unsorted + space) {
            T nextToInsert = a[unsorted];
            index = unsorted - space;
            while ((index >= first) && (nextToInsert.compareTo(a[index]) < 0)) {
                a[index + space] = a[index];
                index = index - space;
            } // end while

            a[index + space] = nextToInsert;
        } // end for
    } // end incrementalInsertionSort

}  // end ShellSort
```

```java
public class ShellSortEnhanced {

    public static <T extends Comparable<? super T>>
            void shellSortEnhanced(T[] a, int n) {
        shellSortEnhanced(a, 0, n - 1);
    } // end betterShellSort

    public static <T extends Comparable<? super T>>
            void shellSortEnhanced(T[] a, int first, int last) {
        int n = last - first + 1; // Number of array elements

        for (int space = n / 2; space > 0; space = space / 2) {
            if (space % 2 == 0) // If space is even, add 1
            {
                space++;
            }

            for (int begin = first; begin < first + space; begin++) {
                incrementalInsertionSort(a, begin, last, space);
            }
        } // end for
    } // end betterShellSort
```

```
442     private static <T extends Comparable<? super T>>
443             void incrementalInsertionSort(T[] a, int first, int last, int space) {
444         int unsorted, index;
445
446         for (unsorted = first + space; unsorted <= last;
447                 unsorted = unsorted + space) {
448             T nextToInsert = a[unsorted];
449             index = unsorted - space;
450             while ((index >= first) && (nextToInsert.compareTo(a[index]) < 0)) {
451                 a[index + space] = a[index];
452                 index = index - space;
453             } // end while
454
455             a[index + space] = nextToInsert;
456         } // end for
457     } // end incrementalInsertionSort
458 }  // end ShellSortEnhanced
459
460 ---------------------------------------------------------------------------
461 // Please change line #480 to try other sorting methods
462
463
464
465 public class SelectionSortDriver {
466
467     private static final Name[] ITEMS
468             = {
469                 new Name("Zeke", "Clayton"), new Name("Bob", "Clayton"),
470                 new Name("Rob", "Smith"), new Name("Ali", "Babba"),
471                 new Name("Jamie", "Perfect"), new Name("Jody", "Perfect"),
472                 new Name("Jim", "Smith"), new Name("John", "Clayton"),
473                 new Name("Bill", "Smith"), new Name("Jamie", "Perfect"),
474                 new Name("Zeke", "Clayton"), new Name("Mickey", "Mouse")
475             };
476
477     public static void main(String[] args) {
478         int n = ITEMS.length;
479         display(ITEMS, n);
480         SelectionSort.selectionSort(ITEMS, n); // SELECTION SORT
481         System.out.println("Is sorted: " + isSorted(ITEMS, n) + "\n");
482         display(ITEMS, n);
483     }  // end main
484
485     public static void display(Object[] array, int n) {
486         for (int index = 0; index < n; index++) {
487             System.out.println(array[index]);
488         }
489         System.out.println();
490     } // end display
491
492     public static boolean isSorted(Name[] array, int n) {
493         boolean sorted = true;
494         for (int index = 0; sorted && (index < n - 1); index++) {
495             if (array[index].compareTo(array[index + 1]) > 0) {
496                 sorted = false;
497             }
498         } // end for
499         return sorted;
500     } // end isSorted
501 }  // end Driver
502
503
504
```

```
Ali Babba
Bob Clayton
John Clayton
Zeke Clayton
Zeke Clayton
Mickey Mouse
Jamie Perfect
Jamie Perfect
Jody Perfect
Bill Smith
Jim Smith
Rob Smith
```